

Process Synchronization

Module 4

Synchronization

- The process of ensuring that multiple processes are coordinated and do not interfere with each other.
- This is important because processes can share data and resources, and if they are not synchronized, they can overwrite each other's data or cause other problems.

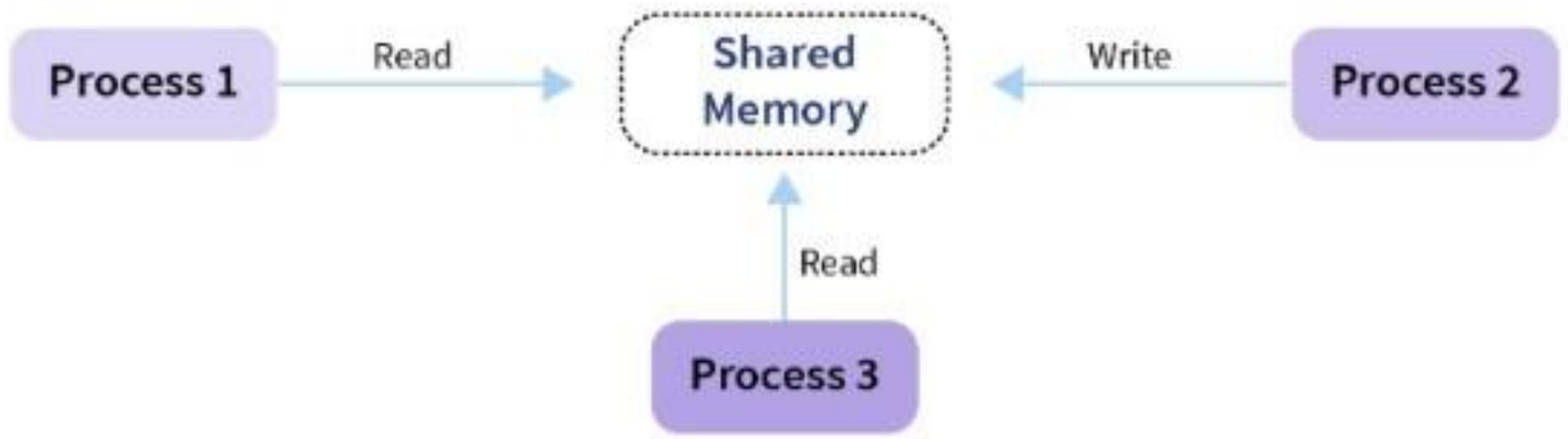
Synchronization - Objective

- The main objective of process synchronization is to
- Ensure that **multiple processes access shared resources** without interfering with each other and
- To prevent the **possibility of inconsistent data** due to concurrent access.

Synchronization

- It is the way by which processes that share the same memory space are managed in an operating system.
- It helps maintain the consistency of data by using variables or hardware so that only one process can make changes to the shared memory at a time.
- Inconsistency of data can occur when various processes share a common resource in a system which is why there is a need for process synchronization in the operating system.

Synchronization – How it Works?



Synchronization – How it Works?

- If a process1 is trying to read the data present in a memory location while another process2 is trying to change the data present at the same location, there is a high chance that the data read by the process1 will be incorrect.

Elements/Sections of a Program/Process

- **Entry Section:** The entry Section decides the entry of a process.
- **Critical Section:** The Critical section allows and makes sure that only one process is modifying the shared data.
- **Exit Section:** The entry of other processes in the shared data after the execution of one process is handled by the Exit section.
- **Remainder Section:** The remaining part of the code which is not categorized as above is contained in the Remainder section.

Critical Section Problem - Primitives

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```


Requirements for Synchronization

- **Mutual exclusion:** If a process is running in the critical section, no other process should be allowed to run in that section at that time.
- **Progress:** If no process is still in the critical section and other processes are waiting outside the critical section to execute, then any one of the threads must be permitted to enter the critical section. The decision of which process will enter the critical section will be taken by only those processes that are not executing in the remaining section.

Requirements for Synchronization

- **No starvation:** Starvation means a process that keeps waiting forever to access the critical section but never gets a chance. No starvation is also known as **Bounded Waiting**.
 - A process should **not wait forever to** enter inside the critical section.
 - When a process submits a request to access its critical section, there should be a limit or bound, **which is the number of other processes that are allowed to access the critical section before it.**
 - After this bond is reached, this process should be allowed to access the critical section.

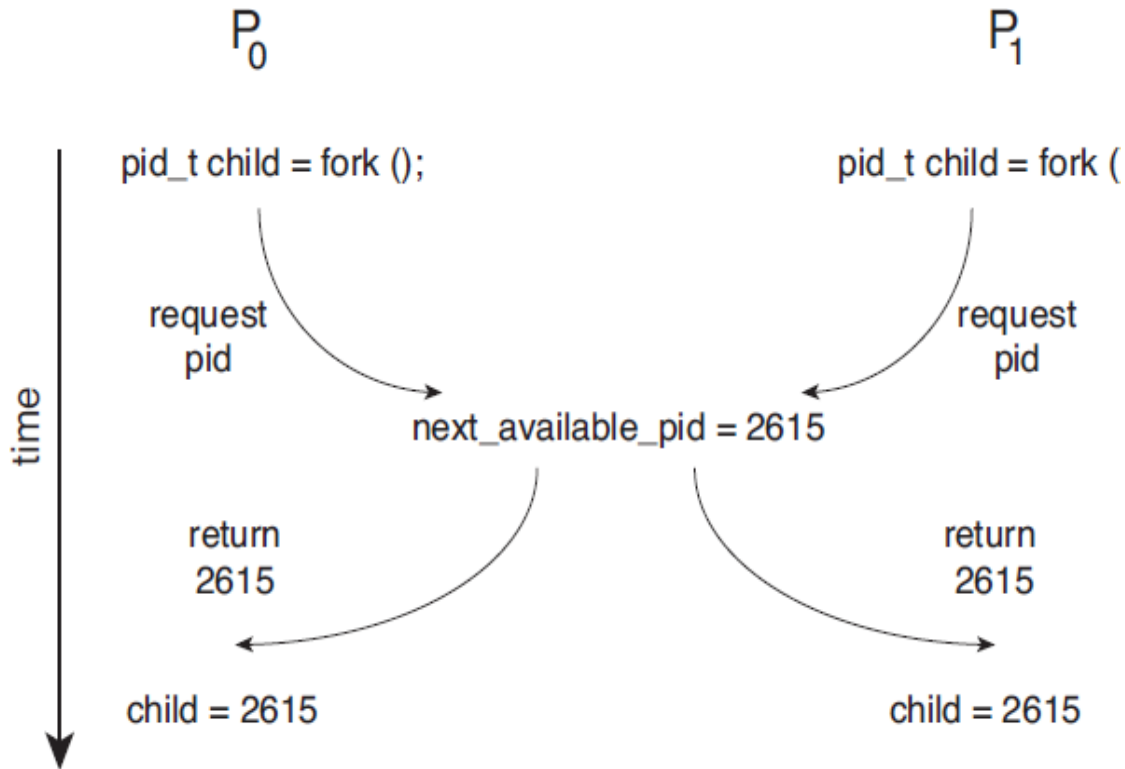
Race Condition

- When more than one process is either running the same code or modifying the same memory or any shared data, there is a risk that the result or value of the shared data may be incorrect because all processes try to access and modify this shared resource.
- Thus, all the processes race to say that my result is correct. This condition is called the race condition.

Race Condition

- Since many processes use the same data, the results of the processes may depend on the **order of their execution** (**Sequence of their execution**)
- In the critical section, a race condition occurs when the **end result of multiple thread executions varies** depending on the **sequence** in which the threads execute.

Race Condition Example



- Processes P_0 and P_1 are creating child processes using the `fork()` system call
- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)
- Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable `next_available_pid` the same pid could be assigned to two different processes!

Critical Section Problem

- When multiple processes need to access shared resources or data, it's essential to ensure that only one process can access the resource at a time. This is known as mutual exclusion.

Critical Section Problem

- A **part of code** that can **only be accessed by a single process at any moment** is known as a **critical section**.
- This means that when a **lot of programs want to access and change a single shared data**, only one process will be allowed to change at any given moment.
- The other processes must wait until the data is free to be used.

Critical Section Problem – What is the Problem?

- A critical section is a code segment that can be accessed by only one process at a time. (How to do it?)
- Designing a way for cooperative processes to access shared resources without creating data inconsistencies.

Critical Section Problem – Solution 1

Peterson's solution

- a classical **software-based** solution.
- The solution is based on the idea that when a process is executing in a critical section, then the other process executes the rest of the code and vice-versa.
- This solution makes sure that only one process executes the critical section at any point in time.

Critical Section Problem – Solution 1

Peterson's solution

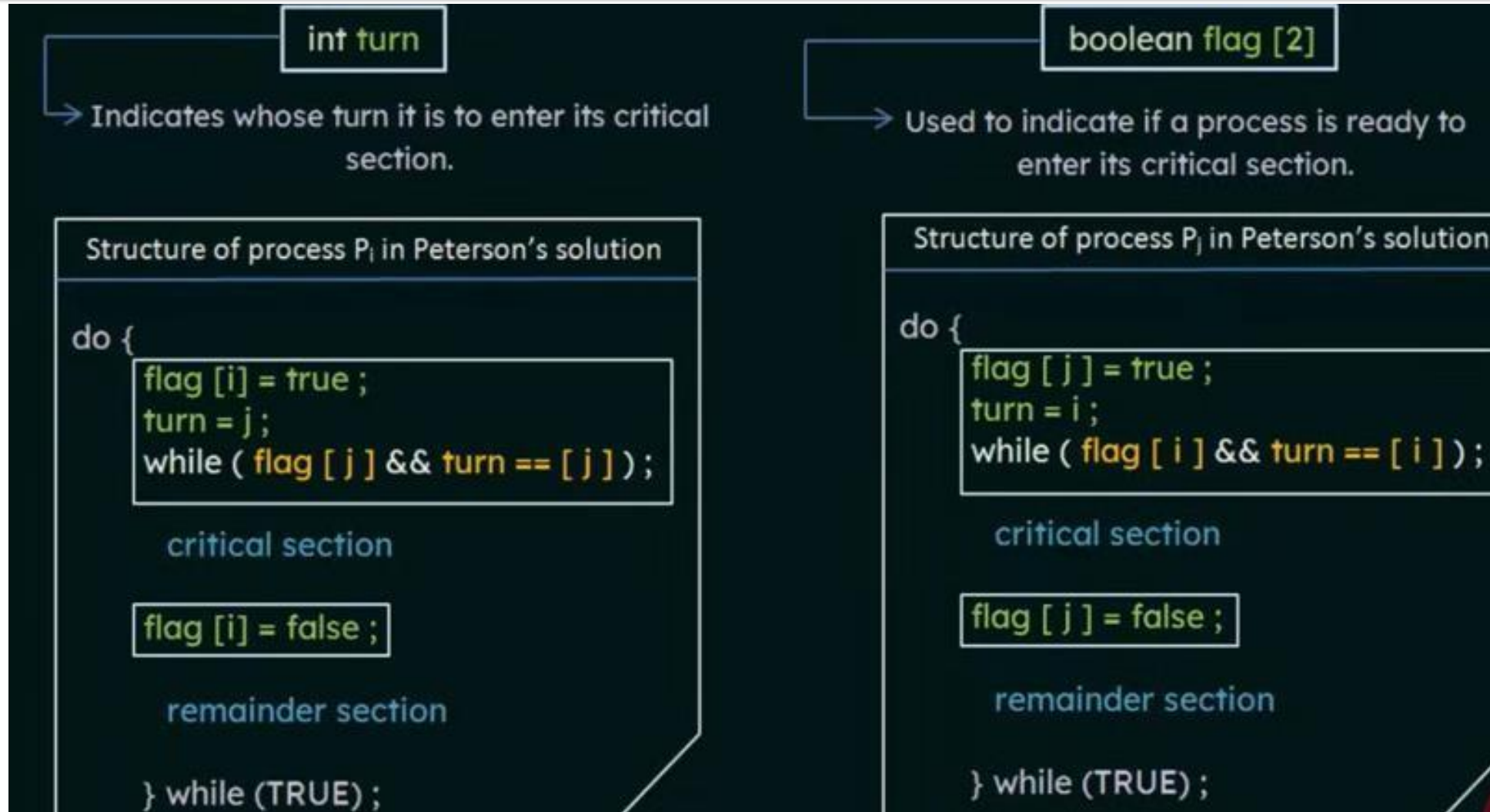
- Two shared variables that are used by the processes.
- **A Boolean Flag[]:** A Boolean array Flag which is initialized to FALSE. This Flag array represents which process is which process wants to enter into the critical section.
- **int Turn:** An integer variable Turn indicates the process number that is ready to enter into the critical section.

Critical Section Problem – Solution 1

Peterson's solution

- Two process solution
- Assume that the **load** and **store** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
 - **int turn;**
 - **boolean flag[2]**
- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section.
 - **flag[i] = true** implies that process **P_i** is ready!

Critical Section Problem – Solution 1



Critical Section Problem – Solution 1

```

while (true) {
    flag[i] = true;

    turn = j; //giving priority to Pj (the jth
process) in case Pj is also trying to enter the
critical section.

    while (flag[j] && turn == j) {

        // Busy wait until process j's flag is false
or turn is i

    }

    // Critical Section (Pi)

    flag[i] = false; // Process i has exited CS

```

```

while (true) {

    flag[j] = true;

    turn = i;

    while (flag[i] && turn == i) {

        // Busy wait until process i's flag
is false or turn is j

    }

    // Critical Section (Pj)

    flag[j] = false; // Process j has exited CS

```

Critical Section Problem – Solution 1

Peterson's solution

- Each process has a flag variable (e.g., flag[0] for P0 and flag[1] for P1).
- The flag[i] is set to true when process P_i wants to enter the critical section, signaling its intent.
- A turn variable is used to decide which process should proceed if both processes are trying to enter the critical section simultaneously.
- This ensures that the other process gets a fair chance to access the critical section.

Critical Section Problem – Solution 1

Peterson's solution

- Before Entering the Critical Section:
 - A process P_i sets $\text{flag}[i] = \text{true}$, indicating it wants to enter the critical section.
 - It then sets $\text{turn} = j$, where j is the index of the other process (e.g., if $i = 0$, then $j = 1$).
 - The process then enters a busy-wait loop: `while (flag[j] && turn == j).`

Critical Section Problem – Solution 1

Peterson's solution

- Before Entering the Critical Section:
 - This loop will only exit if the other process P_j is not interested in entering the critical section ($\text{flag}[j] = \text{false}$) or if the current process P_i is allowed to enter because the turn is in its favor ($\text{turn} \neq j$).

Critical Section Problem – Solution 1

Peterson's solution

- Critical Section:

- Once the loop exits, process P_i enters the critical section. At this point, P_i can safely access shared resources without interference from P_j .

Critical Section Problem – Solution 1

Peterson's solution

- Exiting Critical Section:
 - After finishing the work in the critical section, process P_i sets $\text{flag}[i] = \text{false}$, indicating it no longer needs access to the critical section. This action allows the other process P_j to proceed if it is waiting.

Critical Section Problem – Solution 1

Peterson's solution

- Boolean **flag** and **turn** are both shared variable
- Initially, the flags are false.
- When a process wants to execute its critical section, it sets its
 - **flag** to true and
 - **turn** into the index of the other process
 - This means that the process wants to execute but it will allow the other process to run first.

Critical Section Problem – Solution 1

Peterson's solution

- Set `flag[i]` to true to indicate its intention to enter the critical section.
- Set `turn` to `j` to yield the right to enter the critical section to process `j`.
- Enter a busy waiting loop, checking if `flag[j]` is true and if `turn` is `j`. If both conditions are met, the process continues waiting.
- Once the conditions are no longer met, the process enters its critical section.
- After exiting the critical section, set `flag[i]` to false to indicate that it has finished.

Critical Section Problem – Solution 1

Peterson's solution

- The process performs **busy waiting** until the other process has finished its own critical section.
- After this, the current process enters its critical section
- After completing the critical section, it sets its own flag to false, indicating it does not wish to execute anymore

Critical Section Problem – Solution 1

Peterson's solution

Busy Waiting:

- Busy waiting is a technique in which a process repeatedly checks a condition to enter its critical section instead of being blocked.
- Busy waiting occurs in the while loops where each process checks the flag and turn variables to determine if it's safe to enter its critical section.
- During busy waiting, the process consumes CPU cycles while it waits for the condition to become true.

Critical Section Problem – Solution 1

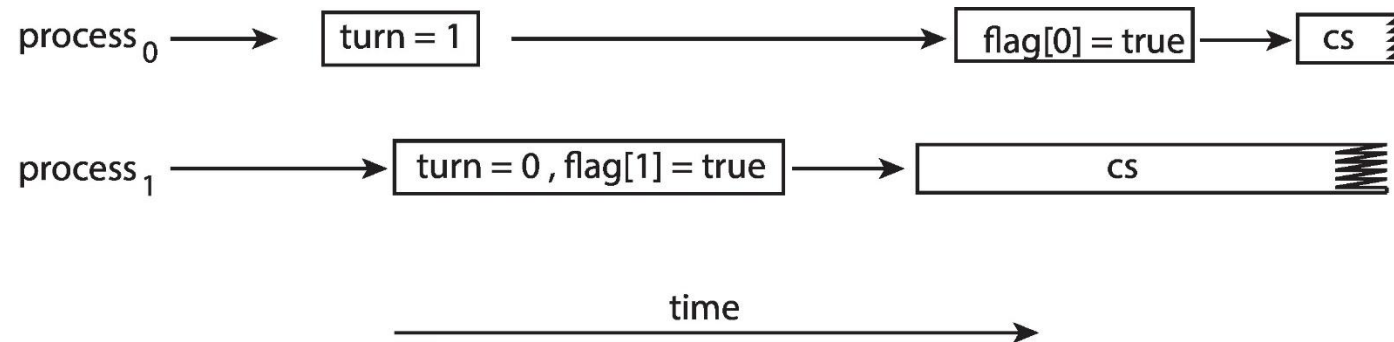
Peterson's solution – Points

- The turn variable acts as a tie-breaker when both processes want to enter the critical section at the same time.
- If both processes set their respective flag to true (indicating their intent to enter the critical section), the turn variable helps to decide which process gets to enter first.

Critical Section Problem – Solution 1

Peterson's Solution Revisited

The effects of instruction reordering in Peterson's Solution



This allows both processes to be in their critical section at the same time!

To ensure that Peterson's solution will work correctly on modern computer architecture we must use **Memory Barrier**.

Critical Section Problem – Solution 1

Memory Barrier

Memory model are the memory guarantees a computer architecture makes to application programs.

Memory models may be either:

- **Strongly ordered** – where a memory modification of one processor is immediately visible to all other processors.
- **Weakly ordered** – where a memory modification of one processor may not be immediately visible to all other processors.

A **memory barrier** is an instruction that forces any change in memory to be propagated (made visible) to all other processors.

Critical Section Problem – Solution 2

Bakery Algorithm

- Designed to ensure that only one process can access a critical section at a time.
- It's particularly useful in distributed systems where processes might be running on different machines.

Critical Section Problem – Solution 2

Bakery Algorithm

- Each process is assigned a unique ticket number. These numbers are ordered lexicographically, meaning they are compared based on their individual digits from left to right.
- Before entering the critical section, a process acquires a ticket number. This number is determined by finding the maximum ticket number among all other processes and then incrementing it.

Critical Section Problem – Solution 2

Bakery Algorithm

- A process waits until its ticket number is the smallest among all processes that are currently trying to enter the critical section or have already entered.
- Once a process has the smallest ticket number, it can enter the critical section.
- After exiting the critical section, the process releases its ticket, allowing other processes to acquire tickets and enter the critical section.

Critical Section Problem – Solution 2

Bakery Algorithm - Example

- Consider three processes, P1, P2, and P3.
- Initial State: All processes have their tickets set to 0.
- P1 Acquires Ticket: P1 acquires ticket number 1.
- P2 Acquires Ticket: P2 acquires ticket number 2.
- P3 Acquires Ticket: P3 acquires ticket number 3.

Critical Section Problem – Solution 2

Bakery Algorithm - Example

- Now, P1 has the smallest ticket number and can enter the critical section.
- After P1 finishes and releases its ticket,
- P2 has the smallest ticket number and can enter.
- Finally, P3 will enter.

Critical Section Problem – Solution 2

Simplified Bakery Algorithm

- Processes numbered 0 to N-1
- num is an array N integers (initially 0).
 - Each entry corresponds to a process

```
lock(i){  
    num[i] = MAX(num[0], num[1], ..., num[N-1]) + 1  
    for(p = 0; p < N; ++p){  
        while (num[p] != 0 and num[p] < num[i]);  
    }  
}
```

critical section

```
unlock(i){  
    num[i] = 0;  
}
```

P1	P2	P3	P4	P5
0	0	0	0	0

Original Bakery Algorithm

- Without atomic operation assumptions
- Introduce an array of N Booleans: *choosing*, initially all values False.

```
lock(i){  
    choosing[i] = True  
    num[i] = MAX(num[0], num[1], ..., num[N-1]) + 1  
    choosing[i] = False  
    for(p = 0; p < N; ++p){  
        while (choosing[p]);  
        while (num[p] != 0 and (num[p],p) < (num[i],i));  
    }  
}
```

doorway

critical section

```
unlock(i){  
    num[i] = 0;  
}
```

P1	P2	P3	P4	P5
0	3	1	2	2

$(a, b) < (c, d)$ which is equivalent to: $(a < c)$ or $((a == c) \text{ and } (b < d))$

Critical Section Problem – Solution 2

```
repeat
    choosing[i] := true;
    number[i] := max(number[0], number[1], ..., number[n-1]) + 1;
    choosing[i] := false;
    for j := 0 to n - 1 do
        begin
            while choosing[j] do no-op; // Wait until process j finishes choosing
its number
            while number[j] != 0 and (number[j], j) < (number[i], i) do no-op; //
Wait until it's i's turn
        end;
        // Critical section
        number[i] := 0; // Exit the critical section
        // Remainder section
    until false;
```


Critical Section Problem – Solution 2

Bakery Algorithm - Points

- **Fairness:** It ensures that all processes eventually get to enter the critical section, preventing starvation.
- **Freedom from Starvation:** No process is indefinitely prevented from entering the critical section.
- **Freedom from Deadlock:** The algorithm prevents deadlocks by ensuring that processes always make progress.
- **Distributed:** It can be used in distributed systems where processes may be running on different machines.

Critical Section Problem – Solution 2

Bakery Algorithm - Points

- Choosing Array (choosing[i]):
 - This is a boolean array where choosing[i] is set to true when process i is selecting a number (a "ticket") for itself before entering the critical section. It is set to false once the process has chosen its number

Critical Section Problem – Solution 2

Bakery Algorithm - Points

- Number Array (number[i]):
 - This array holds the number (or "ticket") for each process. The number represents the process's place in line to enter the critical section.
 - A higher number means the process will wait longer, and a lower number means it gets to enter sooner.

Critical Section Problem – Solution 2

Bakery Algorithm - Points

- **Choosing the number or Ticket**
- $\text{choosing}[i] := \text{true};$ Process i sets its choosing flag to true indicating that it's in the process of choosing a number.
- $\text{Number}[i] := \max(\text{number}[0], \text{number}[1], \dots, \text{number}[n-1]) + 1;$ Process i picks a number that is one greater than the maximum number in the number array. This step ensures that each process gets a unique and sequential number.
- $\text{choosing}[i] := \text{false};$ Process i sets its choosing flag to false after it has chosen its number

Critical Section Problem – Solution 2

Bakery Algorithm - Points

- **Waiting to Enter the Critical Section:**
- for $j := 0$ to $n - 1$: Process i checks all other processes j .
- while choosing[j] do no-op;: Process i waits until process j finishes choosing its number. This ensures that if j is in the middle of choosing its number, i does not make a decision based on incomplete information.

Critical Section Problem – Solution 2

Bakery Algorithm - Points

- **Waiting to Enter the Critical Section:**
- while $\text{number}[j] \neq 0$ and $(\text{number}[j], j) < (\text{number}[i], i)$ do no-op;
 - If process j has a lower number (indicating it has higher priority) and j has not yet entered the critical section ($\text{number}[j] \neq 0$), process i will wait.
 - If $\text{number}[j] == \text{number}[i]$, the process with the smaller index j gets priority. This ensures fairness and prevents deadlock.

Critical Section Problem – Solution 2

Bakery Algorithm - Points

■ Entering the Critical Section:

- Once process i has confirmed that it has the smallest number (or highest priority) among all competing processes, it proceeds to enter the critical section.

■ Exiting the Critical Section:

- $\text{number}[i] := 0$; After finishing in the critical section, process i resets its number to 0, indicating that it no longer wishes to enter the critical section.

Critical Section Problem – Solution 2

Bakery Algorithm - Points

- Remainder Section:
 - The process can then perform other non-critical operations before repeating the process if it needs to enter the critical section again.

Critical Section Problem – Solution 2

Bakery Algorithm - Points

- Simplest known solutions to the mutual exclusion problem for n processes.
- Ensures that shared resources are used efficiently in a multithreaded environment.
- It is free from starvation.
- It uses FIFO
- It works with atomic registers.
- It is unreliable because any one of the processes can fail and halt progress.

Critical Section Problem – Solution 3

Synchronization Hardware

- Hardware-based solutions for critical sections leverage specialized instructions provided by the processor to ensure mutual exclusion. These instructions are atomic, meaning they execute as a single, indivisible operation, preventing other processes from interfering.
- Hardware-based solutions for critical sections leverage specialized instructions provided by the processor to ensure mutual exclusion.
- These instructions are atomic, meaning they execute as a single, indivisible operation, preventing other processes from interfering.

Critical Section Problem – Solution 3

Synchronization Hardware

- Special hardware instructions that allow us to either *test-and-modify* the content of a word, or to *swap* the contents of two words atomically (uninterruptedly.)
 - **Test-and-Set** instruction
 - **Compare-and-Swap** instruction

Critical Section Problem – Solution 3

Synchronization Hardware 1 – Test and Set

- It's a locking mechanism that prevents multiple processes from accessing the same resource
- A shared Boolean variable, commonly known as a 'lock', is used.
- When a process wants to access a resource, it 'tests' the lock. If the lock is **false (unlocked)**, the process sets it to **true (locked)** and proceeds with its task. If the lock is true, the process waits until it's released.

Critical Section Problem – Solution 3

Synchronization Hardware 1 – Test and Set

```
boolean lock = false;
boolean TestAndSet(boolean &target){
boolean returnValue = target;
    target = true;
    return returnValue;
}
while(1){
    while(TestAndSet(lock)); CRITICAL SECTION CODE;
    lock = false;
    REMAINDER SECTION CODE;
}
```

Shared boolean variable lock, initialized to false

Solution:

```
do {
    while (test_and_set(&lock))
        ; /* do nothing */

        /* critical section */

    lock = false;
        /* remainder section */
} while (true);
```

Critical Section Problem – Solution 3

Synchronization Hardware 1 – Test and Set

Thread 1:

- Enters the outer while(1) loop.
- Enters the inner while(TestAndSet(lock)) loop.
- TestAndSet sets lock to true and returns false, exiting the inner loop.
- Executes the CRITICAL SECTION CODE.
- Sets lock to false.
- Executes the REMAINDER SECTION CODE.
- Goes back to the beginning of the outer loop.

Synchronization Hardware 1 – Test and Set

Thread 2:

- Enters the outer while(1) loop.
- Enters the inner while(TestAndSet(lock)) loop.
- TestAndSet sees that lock is already true and returns true, continuing the inner loop.
- Thread 2 will keep spinning in this loop until Thread 1 releases the lock.

Critical Section Problem – Solution 3

Synchronization Hardware 1 – Test and Set

- The target parameter in the function is passed by reference, meaning that any changes made to target within the function will also affect the original variable that was passed to the function. In this case, the original variable is lock.

Critical Section Problem – Solution 3

Synchronization Hardware 2 – Compare and Swap

- It involves two operations: 'swap' and 'unlock'.
- A boolean lock variable is used here as well.
- The 'swap' operation exchanges the value of the lock with a local variable.
- If the lock was false, it becomes true, and the process proceeds.
- The 'unlock' operation sets the lock back to false when the process is done.

Critical Section Problem – Solution 3

Synchronization Hardware 2 – Compare and Swap

```
boolean lock = false;
individual key = false;
```

```
void swap(boolean &a, boolean &b){
    boolean temp = a;
    a = b;
    b = temp;
}
```

```
while(1){
    key=true;
    while(key){
        swap(lock,key);
    }
}
```

CRITICAL SECTION CODE

lock = false;

REMAINDER SECTION CODE

Shared integer lock initialized to 0;

Solution:

```
while (true){
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */

    /* critical section */

    lock = 0;

    /* remainder section */
}
```

Critical Section Problem – Solution 3

Synchronization Hardware – Compare and Swap

- Each process executes the following loop:
 - Sets its **key to true**, indicating its intention to enter the critical section.
 - While **key** is still true, repeatedly swaps the values of **lock** and **key**.
 - If the **swap operation is successful** (i.e., **lock** was false and **key** was true), the process has acquired the lock and can enter the critical section.
 - After executing the critical section, the process sets **lock** to false to **release the lock** for other processes.
 - The loop continues, allowing the process to try entering the critical section again if necessary.

Critical Section Problem – Solution 3

Synchronization Hardware – Compare and Swap

- The `compare_and_swap` instruction attempts to atomically update the lock variable.
- If the lock is currently 0, it is set to 1, indicating that the process has entered the critical section.
- If the lock is already 1, the process retries the loop until it successfully acquires the lock.

Critical Section Problem – Solution 3

Synchronization Hardware – Compare and Swap

- Variables:
- **key:** A boolean variable for each process, indicating whether the process is trying to enter the critical section.
- **lock:** A shared boolean variable representing the lock for the critical section.

Critical Section Problem – Solution 3

Atomic Variables

- Typically, instructions such as compare-and-swap are used as building blocks for other synchronization tools.
- One tool is an **atomic variable** that provides *atomic* (uninterruptible) updates on basic data types such as integers and booleans.
- For example:
 - Let **sequence** be an atomic variable
 - Let **increment()** be operation on the atomic variable **sequence**

The Command:

- **increment(&sequence) ;**
- ensures **sequence** is incremented without interruption

Critical Section Problem – Solution 3

Atomic Variables

The `increment()` function can be implemented as follows:

```
void increment(atomic_int *v)
{
    int temp;
    do {
        temp = *v;
    }
    while (temp != (compare_and_swap(v, temp, temp+1)) );
}
```

Process Synchronization – Solution 4

Semaphores

- Normal variables used to coordinate the activities of multiple processes
- Used to enforce mutual exclusion, avoid race conditions, and implement synchronization between processes.
- How different processes (or programs) share resources, like memory or data, without causing conflicts.

Process Synchronization – Solution 4

Semaphores

- Semaphore proposed by Edsger Dijkstra, is a technique to manage concurrent processes by using a simple integer value, which is known as a semaphore.
- Semaphore is simply a variable which is non-negative and shared between threads. This variable is used to solve the critical section problem and to achieve process synchronization in the multiprocessing environment.
- A semaphore **S** is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: **wait ()** and **signal ()**.

wait () → **P** [from the Dutch word **proberen**, which means "to test"]

signal () → **V** [from the Dutch word **verhogen**, which means "to increment"]

Process Synchronization – Solution 4

Semaphores

Semaphore S – integer variable

Can only be accessed via two indivisible (atomic) operations

`wait()` and `signal()` → Originally called `P()` and `V()`

Definition of the `wait()` operation

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

Definition of the `signal()` operation

```
signal(S) {  
    S++;  
}
```

Process Synchronization – Solution 4

Definition of wait ():

```
P (Semaphore S) {  
    while (S <= 0)  
        ; // no operation  
    S-- ;  
}
```

Definition of signal ():

```
V (Semaphore S) {  
    S++ ;  
}
```

All the modifications to the integer value of the semaphore in the wait () and signal() operations must be executed indivisibly. That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

Types of Semaphores:

1. Binary Semaphore:

The value of a binary semaphore can range only between 0 and 1. On some systems, binary semaphores are known as mutex locks, as they are locks that provide mutual exclusion.

2. Counting Semaphore:

Its value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.

Process Synchronization – Solution 4

Semaphores – Primitives - wait (P) and signal (V).

- The **wait operation** decrements the value of the semaphore, and the **signal operation** increments the value of the semaphore.
- When the value of the semaphore is zero, any process that performs a wait operation will be blocked until another process performs a signal operation.

Process Synchronization – Solution 4

Semaphores – Primitives - wait (P) and signal (V).

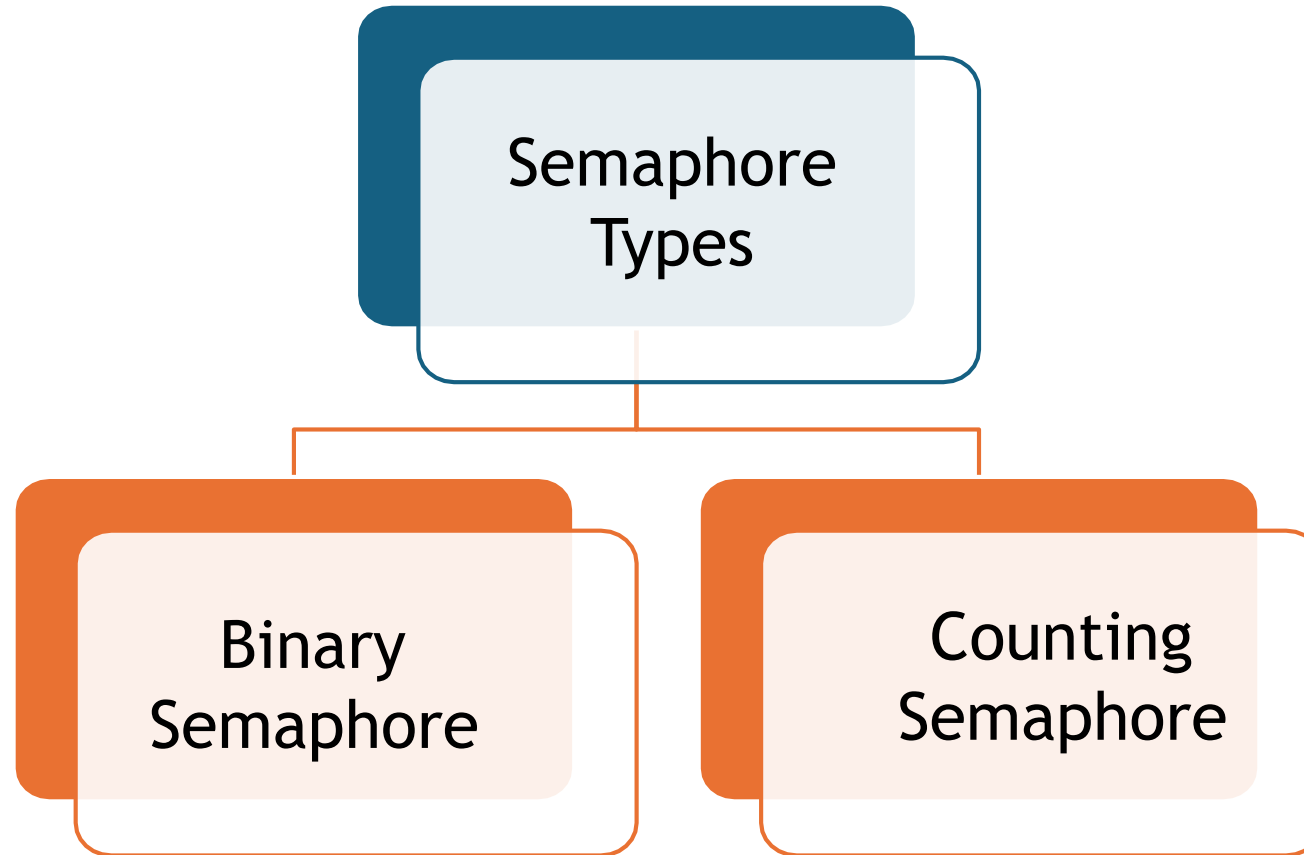
- When a process performs a wait operation on a semaphore, the operation checks whether the value of the semaphore is >0 .
- If so, it decrements the value of the semaphore and lets the process continue its execution;
- otherwise, it blocks the process on the semaphore
- A signal operation on a semaphore activates a process blocked on the semaphore if any or increments the value of the semaphore by 1.
- Due to these semantics, semaphores are also called counting semaphores.

Process Synchronization – Solution 4

Semaphores – Primitives - wait (P) and signal (V).

- **Wait()/Down()/P()**- Helps in controlling the entry of processes in CS. The operation decrements the value of its argument as soon as a process enters the critical section.
- **Signal()/UP()/V()**- Helps control the process's exit after execution from CS. Increment the value of its argument as soon as the execution is finished.

Process Synchronization – Solution 4



09/09/24 **Binary semaphore** – integer value can range only between 0 and 1

Counting semaphore – integer value can range over an unrestricted domain

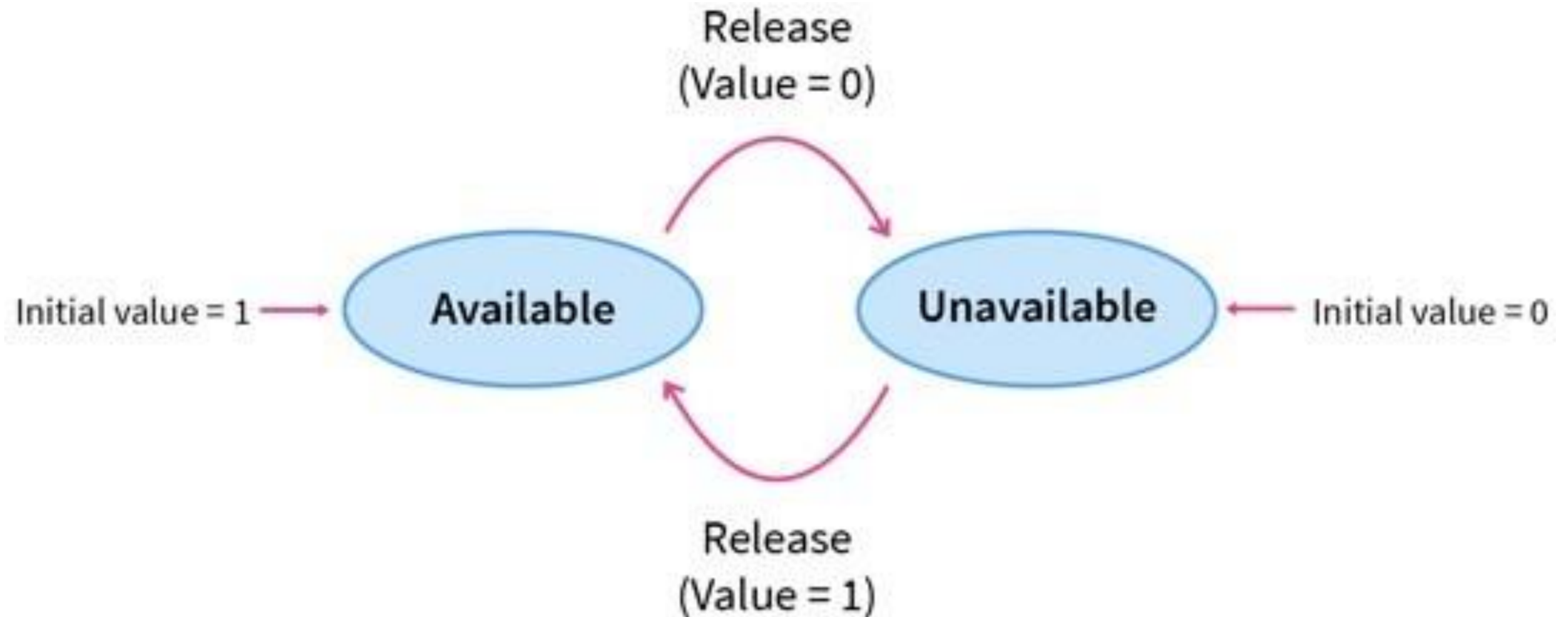
Process Synchronization – Solution 4

Binary Semaphores

- This is also known as a mutex lock.
- It can have only two values 0 and 1.
- Its value is initialized to 1.
- It is used to implement the solution of critical section problems with multiple processes.

Process Synchronization – Solution 4

Binary Semaphores



Process Synchronization – Solution 4

Binary Semaphores

- If the value of the semaphore was initially 1, then on the entering of the process into the critical section, the wait function would have decremented the value to 0 meaning that no more processes can access the critical section (making sure of mutual exclusion -- only in binary semaphores).

State 1 :

P1 → Executing in non critical section
S = 1
P2 → Executing in non critical section

State 2 :

P1 → Enters critical section updates
S = 0, S = 0,
P2 → Executing in non critical section

State 3 :

P1 → Executing in critical section
S = 0
P2 → Wants to enter critical section but
can't as S = 0

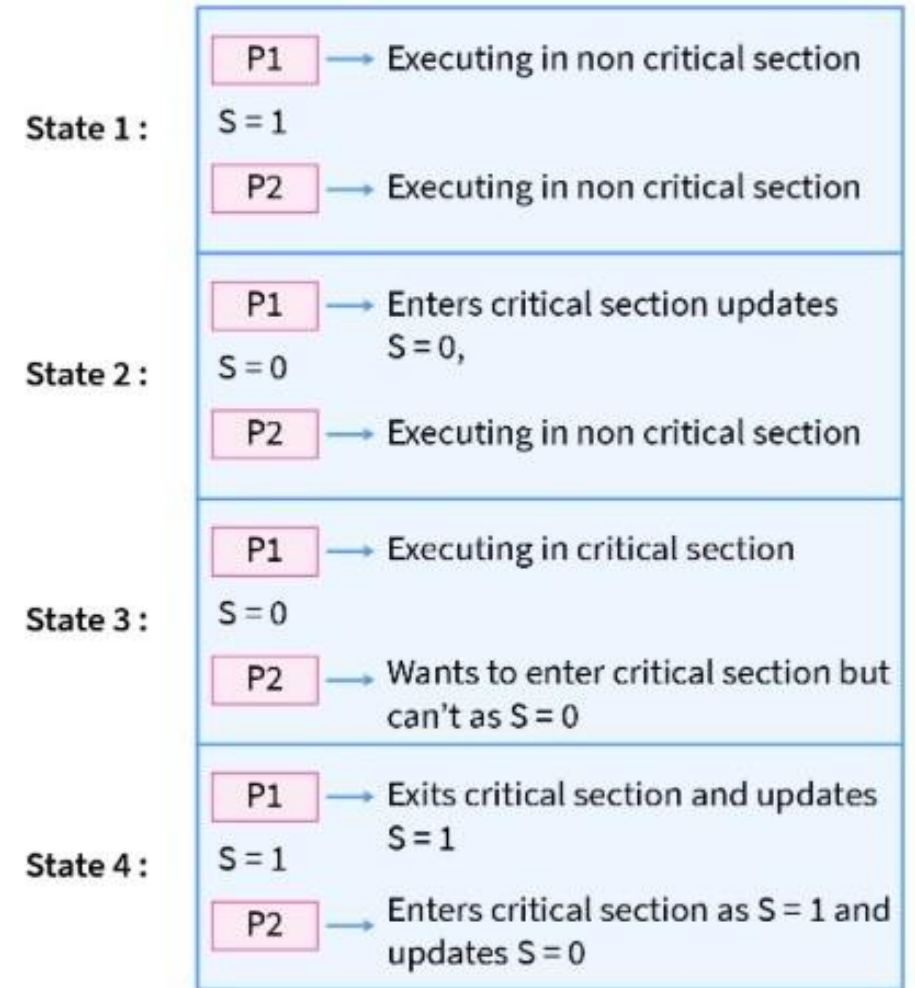
State 4 :

P1 → Exits critical section and updates
S = 1, S = 1
P2 → Enters critical section as S = 1 and
updates S = 0

Process Synchronization – Solution 4

Binary Semaphores

- Once the process exits the critical section, the signal operation is executed and the value of the semaphore is incremented by 1, meaning that the critical section can now be accessed by another process.



Process Synchronization – Solution 4

Binary Semaphores

```
int semaphore = 1;
void P() {
    while (semaphore == 0) {
        // Busy waiting until the semaphore is unlocked
    }
    semaphore = 0;
}
void V() {
    semaphore = 1; // Unlock the semaphore (exit critical section)
}
```

Process Synchronization – Solution 4

Counting Semaphores

- Signaling integers that can take on any integer value
- Coordinate access to resources
- Here the Semaphore count is the number of resources available
- If the value of the Semaphore is anywhere above 0, processes can access the critical section or the shared resources.
- The number of processes that can access the resources/code is the value of the semaphore

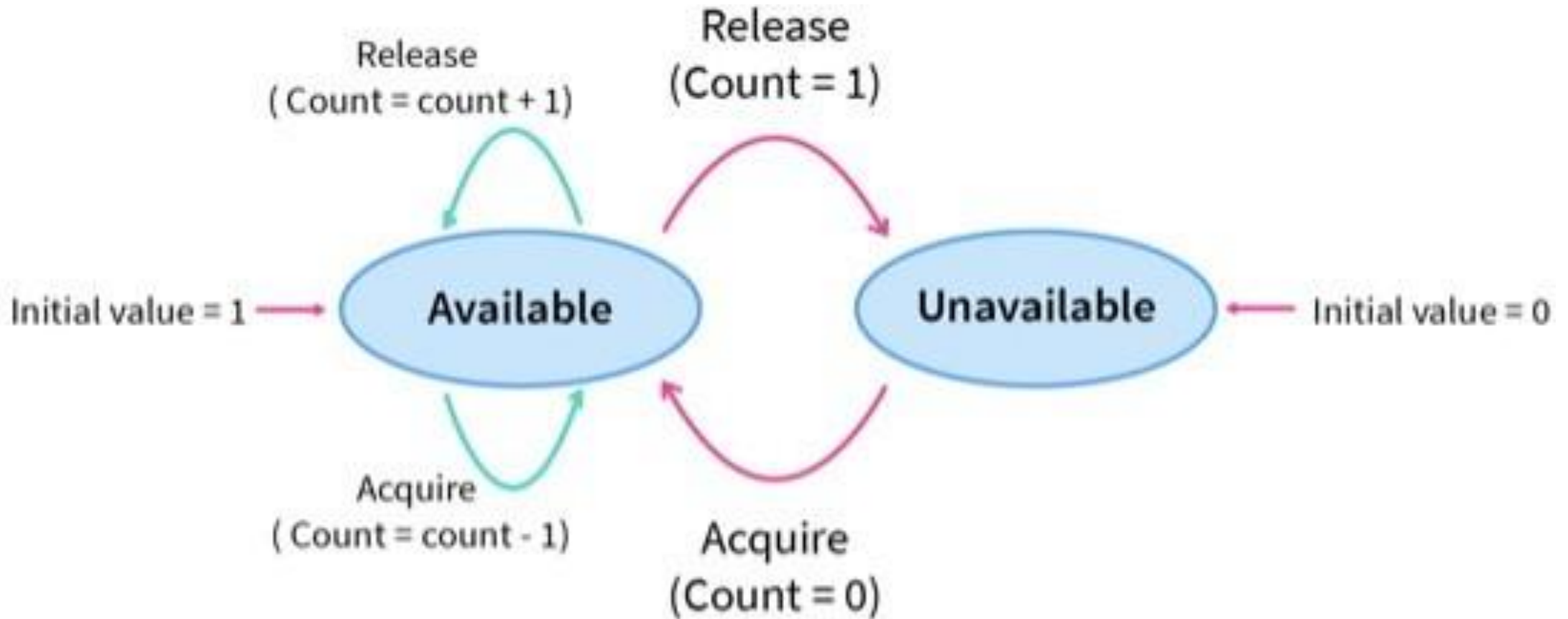
Process Synchronization – Solution 4

Counting Semaphores

- if the value is 0, it means that there aren't any resources that are available, or the critical section is already being accessed by several processes and cannot be accessed by more processes.
- Counting semaphores are generally used when the number of instances of a resource is more than 1, and multiple processes can access the resource.

Process Synchronization – Solution 4

Counting Semaphores



Process Synchronization – Solution 4

Counting Semaphores

- P() (Wait): This decreases the value of the semaphore. If the value is greater than 0, the process can proceed. If it is 0, the process must wait until the value becomes greater than 0.
- V() (Signal): This increases the value of the semaphore, signaling that a resource has been released and is available for other processes

Process Synchronization – Solution 4

Counting Semaphores

```
int semaphore = 3;

void P() {
    while (semaphore == 0) {
        // Busy waiting until a resource becomes available
    }
    semaphore--; // Decrease the semaphore value (acquire a resource)
}

void V() {
    semaphore++; // Increase the semaphore value (release a resource)
}

// Example usage with multiple processes using a shared resource
```


Process Synchronization – Solution 5

Monitors

- Monitors are a programming language component that aids in the regulation of shared data access.
- The Monitor is a package that contains shared data structures, operations, and synchronization between concurrent procedure calls.
- Therefore, a monitor is also known as a synchronization tool

Process Synchronization – Solution 5

Monitors

- Processes operating outside the monitor can't access the monitor's internal variables, but they can call the monitor's procedures.

```
monitor monitor_name{  
    //declaring shared variables  
  
    variables_declaration;  
    condition_variables;  
  
    procedure p1(...) {  
        ...  
    };  
  
    procedure p2(...) {  
        ...  
    };  
    ...  
    procedure pn(...) {  
        ..  
    };  
  
    {  
        Initailisation Code();  
    }  
}
```

Process Synchronization – Solution 5

Monitors

- `monitor_name` is the name of the monitor
- `p1`, `p2`, `pn` are the procedures that provide access to shared data and variables enclosed in monitors
- It ensure that only one thread is accessed at a time for using the shared resource.

Process Synchronization – Solution 5

Syntax of a Monitor

```
monitor monitor_name
{
    // shared variable declarations
    procedure P1 (...) {
        ...
    }
    procedure P2 (...) {
        ...
    }
    .
    .
    procedure Pn (...) {
        ...
    }
    initialization code (...) {
        ...
    }
}
```

- A procedure defined within a monitor can access only those variables declared locally within the monitor and its formal parameters.
- Similarly, the local variables of a monitor can be accessed by only the local procedures.
- The monitor construct ensures that only one process at a time can be active within the monitor.

Condition Construct-

condition x, y;

The only operations that can be invoked on a condition variable are **wait ()** and **signal ()**.

The operation **x.wait()**; means that the process invoking this operation is suspended until another process invokes **x.signal ()**;

The **x. signal ()** operation resumes exactly one suspended process.

Process Synchronization – Solution 5

Monitors - Components

- **Initialization:** Initialization code is executed when a monitor is initialized. It is needed once when the monitors are created and initialized.
- It helps initialize any shared data structures that the monitor will use to perform specific tasks.
- Initialization code is used to initialize a variable when we want to use it.

Process Synchronization – Solution 5

Monitors - Components

- Private Data: It is an essential feature of monitors that helps make the data private.
- It is involved in holding monitors' confidential data, which includes private functions.
- Private fields and functions are not visible outside of the monitor.

Process Synchronization – Solution 5

Monitors - Components

- **Monitor Procedure:** Procedures operate on shared variables defined inside the monitor.
- Monitor procedures or functions can be invoked outside the monitor.
- These are functions or methods which are executed within the monitors' context.
- They are usually used for shared resource manipulation.

Process Synchronization – Solution 5

Monitors - Components

- **Queue:** Queue data structure maintains a list of threads waiting for a shared resource.
- Whenever we request a thread to use shared resources, but another thread is currently using the shared resource, then we cannot use the shared resource at that time.
- But whenever the resource is in its available state, the thread at the front is granted access to the resource.

Process Synchronization – Solution 5

Monitors - Components

- **Condition Variables:** They are used when we have to check for a particular condition before moving on, in conjunction with a mutex, and in providing reliable synchronization.
- Condition variables are the container of threads. These threads wait for a particular condition. Thus, they determine if a specific condition is true or false.
- It can also be considered as a queue or a thread that stops its execution and enters the queue if the specified condition turns out to be false.
- But, if another thread makes the condition accurate, it sends a signal to the thread in front of a queue for continuing execution.

Process Synchronization – Solution 5

Monitors - Components

- **Condition Variables:** Condition variables enable threads to wait for a particular condition and check if it is true before proceeding. The components of condition variables of monitors are mentioned below:
- **Wait() Function:** This function is used for releasing a mutex associated with the monitor.
 - It waits for a particular condition to be valid.
 - A mutex is a program object that enables sharing of the same resources by multiple programs by taking turns.
 - When a thread calls the wait() function, it adds a thread to the list and puts it to sleep.
 - It releases the mutex and enters a blocked state until another thread signals the condition.
 - Therefore, variables are suspended and placed in the condition variables' block queue whenever we perform a wait operation.
- **Signal() Function:** This function is used to wake up a waiting thread.
 - When a thread calls the function, it allies the thread to proceed by waking it up.
 - This function gives a chance to one of the blocked variables.

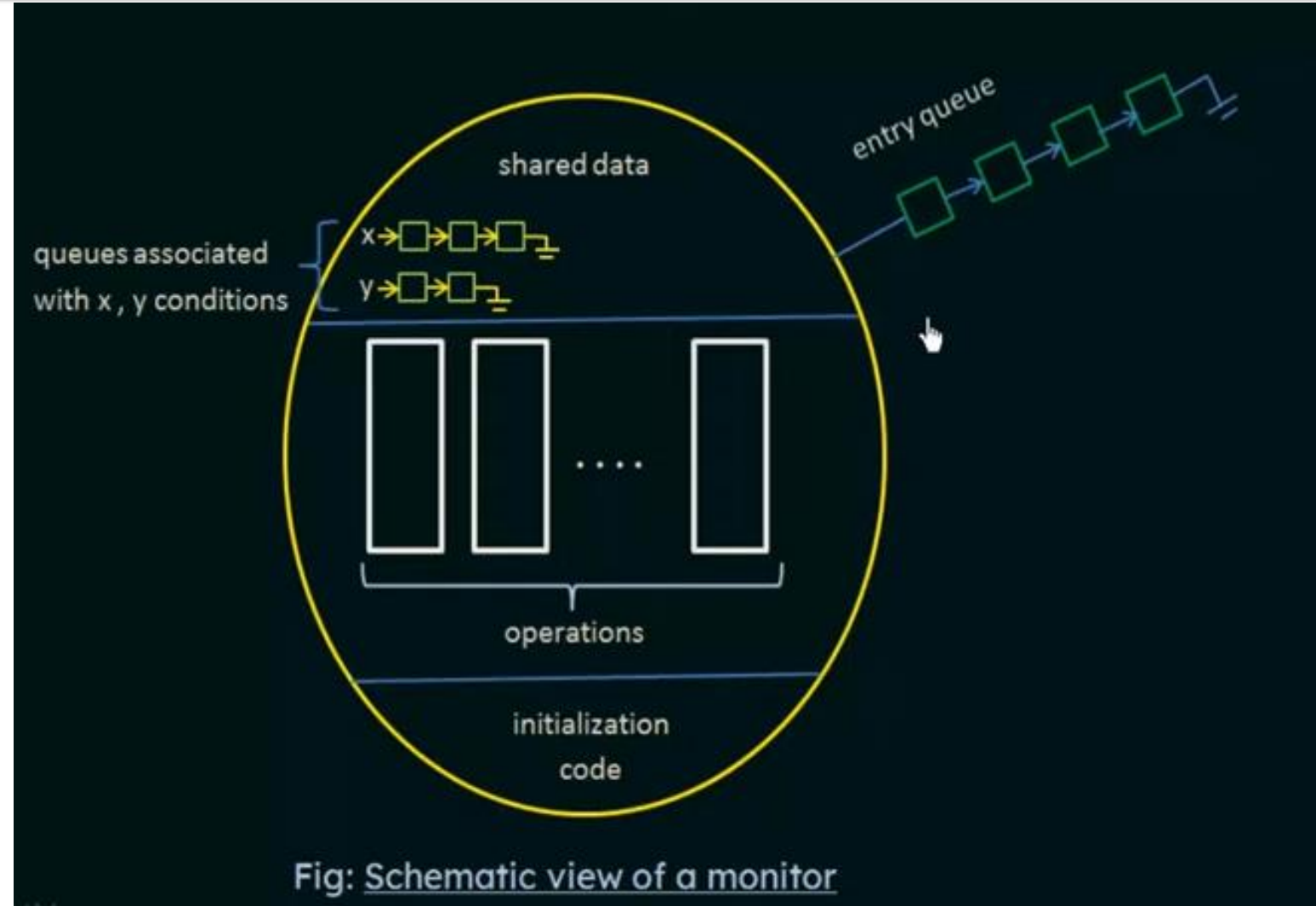
Process Synchronization – Solution 5

Monitors – Overall Working

- When a thread wants to access the shared data, it enters the monitor. If no other thread is using the monitor, the thread proceeds.
- If another thread is already inside the monitor, the new thread is blocked until the monitor becomes available (the current thread exits the monitor).
- Inside the monitor, condition variables can be used to let a thread wait until a specific condition is met (e.g., waiting for input or for a buffer to be filled). When this condition occurs, another thread signals it, allowing the waiting thread to proceed

Process Synchronization – Solution 5

Monitors – Overall Working



Process Synchronization – Solution 6

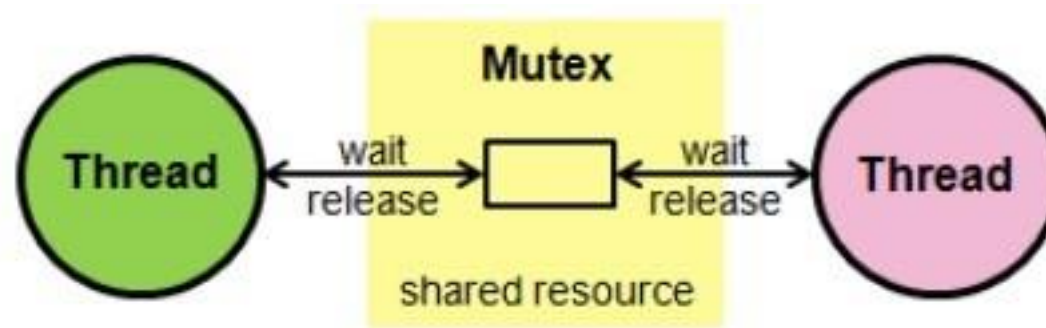
Mutex

- A mutex is a locking mechanism that synchronizes access to a resource.
- The mutex can be acquired by only one process or thread.
- It indicates that a mutex has ownership and only the owner can release the lock.
- Mutex stands for mutual exclusion object.
- Mutex is special a binary semaphore that synchronizes the access to shared resources like memory or I/O.

Process Synchronization – Solution 6

Mutex

- A mutex is a binary variable used to provide a locking mechanism. It offers mutual exclusion to a section of code that restricts only one thread to work on a code section at a given time.



Process Synchronization – Solution 6

Mutex

- **Lock (acquire):** Before a thread can access a shared resource, it must acquire the lock (lock the mutex).
- **Unlock (release):** Once the thread is done using the shared resource, it releases the lock, allowing other threads to access the resource.

Process Synchronization – Solution 6

Mutex

```
int mutex = 1;
void lock() {
    while (mutex == 0) {
        // Busy waiting if the mutex is locked by another thread
    }
    mutex = 0;
}
void unlock() {
    mutex = 1;    // Unlock the mutex
}
void critical_section() {
    lock();    // Acquire the lock before entering the critical section

    // Critical section code (accessing the shared resource)
    printf("Accessing shared resource...\n");

    unlock();    // Release the lock after leaving the critical section
}
```


Process Synchronization – Overview

- **Race Conditions:** It occurs when some common shared resources are accessed by multiple processes or threads, leading to unpredictable behavior.
- **Critical Section:** section of code where shared resources are accessed. Only one process or thread should be allowed to execute the critical section simultaneously.
- **Deadlocks:** a situation occurs when two or more processes are holding the resources and do not release them causing a deadlock to occur because another is waiting for that resources. Deadlock prevention and detection mechanisms are critical in process synchronization.
- **Starvation:** One process is being unable to access resources due to other processes holding onto them for extended periods, often due to priority scheduling or unfair resource allocation.

Process Synchronization – Overview

- **Mutex(Mutual Exclusion):** only one process can access the critical section at a time, it is a locking mechanism.
- **Semaphores:** Signaling mechanisms to control access to shared resources.
- **Monitors:** High-level synchronization constructs that encapsulate shared resources and provide a safe interface for processes to access them.
- **Locks :** Mechanisms to protect shared resources from concurrent access. Locks can be simple or complex